KAM Protocol

Mathematical Specification

and Invariant Analysis

Keyrock Asset Management

March 27, 2026

Abstract

This document provides a rigorous mathematical formalization of the KAM (Keyrock Asset Management) protocol. KAM is an institutional-grade on-chain asset management system implementing a dual-track architecture: institutions interact through a direct 1:1 minting/redemption gateway (`kMinter`), while retail users earn yield via share-based staking vaults (`kStakingVault`). All asset flows are coordinated by a central settlement engine (`kAssetRouter`) using batch processing with timelocked proposals. We derive the core mathematical relationships, formalize the fee model including management fees, performance fees with hurdle rates and high-water marks, define the share accounting with ERC-4626 inflation-attack protection, and enumerate the critical system invariants that must hold for protocol correctness.

Contents

1 Notation and Constants

Definition 1.1 (Protocol Constants)**.** *The following constants parameterize the protocol:*

$$\text{MAX\_BPS} = 10{,}000 \quad \textit{(basis-point denominator; 10 000 bp} = 100\%\textit{)} \tag{1}$$

$$\text{SECS\_PER\_YEAR} = 31{,}556{,}952 \quad \textit{(seconds per year, 365.2425 days)} \tag{2}$$

$$\text{VIRTUAL\_ASSETS} = 10^6 \quad \textit{(virtual asset offset for ERC-4626 protection)} \tag{3}$$

$$\text{VIRTUAL\_SHARES} = 10^6 \quad \textit{(virtual share offset for ERC-4626 protection)} \tag{4}$$

$$T^{\text{default}}_{\text{cooldown}} = 3{,}600 \quad \textit{(default settlement cooldown, 1 hour)} \tag{5}$$

$$T^{\text{max}}_{\text{cooldown}} = 86{,}400 \quad \textit{(maximum settlement cooldown, 1 day)} \tag{6}$$

$$\delta^{\text{default}} = 1{,}000 \quad \textit{(default max yield delta, 10\% in bp)} \tag{7}$$

Definition 1.2 (State Variables)**.** *For a given vault v at time t, we define:*

- $\mathcal{A}_v(t)$*: total gross assets under management*
- $\mathcal{A}_v^{\text{net}}(t)$*: total net assets (after accrued fees)*
- $\mathcal{S}_v(t)$*: total supply of* stkToken *shares*
- $\pi_v(t)$*: share price (assets per share)*
- $\hat{\pi}_v$*: high-water mark share price for performance fees*
- $\Psi_v^+(t)$*: total pending stake requests (kTokens deposited, shares not yet issued)*
- $\Psi_v^-(t)$*: total pending unstake claims (kTokens reserved for withdrawal)*
- $\mathcal{V}(v, a)$*: virtual balance of asset a tracked by the adapter for vault v*
- $\mathcal{G}(v, a)$*: global pending redemption/transfer requests against vault v for asset a*
- d_v*: decimal precision of vault v (typically 6 for USDC, 8 for WBTC)*

2 Token Architecture

The protocol operates a **dual-token system** per supported underlying asset a:

1. **kToken** (kToken_a): A wrapped representation of the underlying asset a. Minted 1:1 upon institutional deposit. The kToken supply reflects both institutional deposits and yield distributed to staking vaults.

2. **stkToken** ($\mathsf{stkToken}_a$): A yield-bearing share token issued by the `kStakingVault`. The share price π_v appreciates as the vault accrues yield. One stkToken represents a proportional claim on the vault's net assets.

Invariant 2.1 (kToken Supply Backing)**.** *At all times, the total kToken supply for asset a equals the sum of all assets under management across all vaults plus any pending operations:*

$$\text{kToken}_a.\text{totalSupply} = \sum_{v \in \mathcal{V}_a} \mathcal{V}(v, a) + \sum_{v \in \mathcal{V}_a^{\text{stk}}} \text{kToken}_a.\text{balanceOf}(v) \tag{8}$$

where $\mathcal{V}_a$ is the set of all vaults holding asset a and $\mathcal{V}_a^{\text{stk}}$ is the subset of staking vaults.

3 Institutional Path: kMinter

3.1 Minting

An institution deposits underlying asset a and receives kToken_a at a 1:1 ratio.

Definition 3.1 (Mint Operation)**.** *Given an institution depositing amount x of asset a into batch b:*

$$\mathsf{kToken}_a.\text{totalSupply} \leftarrow \mathsf{kToken}_a.\text{totalSupply} + x \tag{9}$$
$$b.\text{depositedInBatch} \leftarrow b.\text{depositedInBatch} + x \tag{10}$$
$$\text{totalLockedAssets}[a] \leftarrow \text{totalLockedAssets}[a] + x \tag{11}$$

The underlying assets are transferred to the kAssetRouter and forwarded to the vault adapter.

Invariant 3.1 (1:1 Minting)**.** *For every institutional mint of amount x, exactly x kTokens are created and exactly x units of underlying asset are transferred to the protocol:*

$$\Delta\mathsf{kToken}_a.\text{totalSupply} = \Delta\text{underlying}_a^{\text{deposited}} = x \tag{12}$$

3.2 Redemption (Burn)

Institutional redemption is a **two-phase** process: request followed by claim after settlement.

Definition 3.2 (Burn Request)**.** *When an institution requests redemption of amount x of kToken_a:*

$$b.\text{requestedSharesInBatch} \leftarrow b.\text{requestedSharesInBatch} + x \tag{13}$$
$$\mathcal{G}(\text{kMinter}, a) \leftarrow \mathcal{G}(\text{kMinter}, a) + x \tag{14}$$

The kTokens are transferred to the kMinter contract as escrow (not yet burned).

Definition 3.3 (Burn Execution)**.** *After the batch is settled, the institution claims their underlying assets:*

$$\mathsf{kToken}_a.\text{totalSupply} \leftarrow \mathsf{kToken}_a.\text{totalSupply} - x \tag{15}$$
$$\text{totalLockedAssets}[a] \leftarrow \max\big(\text{totalLockedAssets}[a] - x,\ 0\big) \tag{16}$$

The underlying assets are pulled from the `kBatchReceiver` to the recipient.

3.3 Netting

For each batch b in the kMinter, the **net flow** is:

$$\text{netted}_b = b.\text{depositedInBatch} - b.\text{requestedSharesInBatch} \tag{17}$$

This value is signed: positive means net deposits exceed redemptions, negative means net outflows.

4 Retail Path: kStakingVault

4.1 Total Assets Accounting

Definition 4.1 (Gross Total Assets)**.** *The total gross assets of a staking vault v at time t:*

$$\mathcal{A}_v(t) = \mathsf{kToken}.\text{balanceOf}(v) - \Psi_v^+(t) - \Psi_v^-(t) \tag{18}$$

where Ψ_v^+ represents kTokens deposited by users awaiting share issuance, and Ψ_v^- represents kTokens reserved for users awaiting withdrawal.

Definition 4.2 (Net Total Assets)**.** *The total net assets (user-facing value) deducts accumulated fees:*

$$\mathcal{A}_v^{\text{net}}(t) = \mathcal{A}_v(t) - F_{\text{total}}(t) \tag{19}$$

where $F_{\text{total}}(t)$ is the total accrued but unprocessed fees (see Section 5).

4.2 Share Conversion with Inflation Protection

The protocol employs the **ERC-4626 virtual offset** pattern to prevent inflation attacks.

Definition 4.3 (Share-to-Asset Conversion)**.** *Given shares s, total assets $\mathcal{A}$, and total supply $\mathcal{S}$:*

$$\text{toAssets}(s) = \left\lfloor \frac{s \cdot (\mathcal{A} + \text{VIRTUAL\_ASSETS})}{\mathcal{S} + \text{VIRTUAL\_SHARES}} \right\rfloor \tag{20}$$

Definition 4.4 (Asset-to-Share Conversion)**.** *Given assets a, total assets $\mathcal{A}$, and total supply $\mathcal{S}$:*

$$\text{toShares}(a) = \left\lfloor \frac{a \cdot (\mathcal{S} + \text{VIRTUAL\_SHARES})}{\mathcal{A} + \text{VIRTUAL\_ASSETS}} \right\rfloor \tag{21}$$

Property 4.1 (Inflation Attack Resistance)**.** *The virtual offsets* VIRTUAL_ASSETS = VIRTUAL_SHARES = 10^6 *make inflation attacks economically infeasible. An attacker attempting the classic ERC-4626 donation attack must donate approximately $10^6\times$ the victim's deposit to steal one unit of the victim's funds, making the attack unprofitable.*

Remark 4.1 (Settlement-Time Conversion)**.** *At settlement time, the vault also uses a conversion* without *virtual offsets for accurate batch accounting:*

$$\text{toAssets}_{\text{exact}}(s) = \begin{cases} s & \text{if } \mathcal{S} = 0 \\ \left\lfloor \frac{s \cdot \mathcal{A}}{\mathcal{S}} \right\rfloor & \text{otherwise} \end{cases} \tag{22}$$

$$\text{toShares}_{\text{exact}}(a) = \begin{cases} a & \text{if } \mathcal{S} = 0 \\ \left\lfloor \frac{a \cdot \mathcal{S}}{\mathcal{A}} \right\rfloor & \text{otherwise} \end{cases} \tag{23}$$

These are used in settlement calculations and the kAssetRouter's netting logic for staking vaults.

4.3 Share Price

Definition 4.5 (Gross Share Price)**.**

$$\pi_v^{\text{gross}}(t) = \text{toAssets}\Big(10^{d_v}\Big) = \frac{10^{d_v} \cdot \big(\mathcal{A}_v(t) + \text{VIRTUAL\_ASSETS}\big)}{\mathcal{S}_v(t) + \text{VIRTUAL\_SHARES}} \tag{24}$$

Definition 4.6 (Net Share Price)**.**

$$\pi_v^{\text{net}}(t) = \frac{10^{d_v} \cdot \big(\mathcal{A}_v^{\text{net}}(t) + \text{VIRTUAL\_ASSETS}\big)}{\mathcal{S}_v(t) + \text{VIRTUAL\_SHARES}} \tag{25}$$

The net share price reflects the true user-facing value per stkToken after all fee obligations.

4.4 Staking (Request Phase)

Definition 4.7 (Stake Request)**.** *A user depositing x kTokens into vault v during batch b:*

$$\Psi_v^+ \leftarrow \Psi_v^+ + x \tag{26}$$

$$b.\text{depositedInBatch} \leftarrow b.\text{depositedInBatch} + x \tag{27}$$

$$\mathcal{G}(\text{kMinter}, a) \leftarrow \mathcal{G}(\text{kMinter}, a) + x \tag{28}$$

The kTokens are transferred from the user to the vault. The kAssetRouter is notified via `kAssetTransfer` *to coordinate the virtual balance transfer from kMinter to the staking vault.*

4.5 Unstaking (Request Phase)

Definition 4.8 (Unstake Request)**.** *A user requesting to unstake s stkTokens from vault v during batch b:*

$$b.\text{requestedSharesInBatch} \leftarrow b.\text{requestedSharesInBatch} + s \tag{29}$$

The stkTokens are transferred from the user to the vault as escrow (not burned until settlement). This preserves the share price for all holders during the batch period.

4.6 Netting for Staking Vaults

For staking vaults, the netting computation converts requested shares to asset terms:

$$\text{requestedAssets}_b = \begin{cases} b.\text{requestedSharesInBatch} & \text{if } \mathcal{S}_v = 0 \\ \left\lfloor \dfrac{b.\text{requestedSharesInBatch} \cdot \mathcal{A}_v}{\mathcal{S}_v} \right\rfloor & \text{otherwise} \end{cases} \tag{30}$$

$$\text{netted}_b = b.\text{depositedInBatch} - \text{requestedAssets}_b \tag{31}$$

5 Fee Model

The protocol implements a two-component fee structure: management fees (time-based) and performance fees (return-based with hurdle rate and high-water mark).

5.1 Management Fees

Definition 5.1 (Management Fee)**.** *Given a management fee rate f_m (in basis points), the management fee accrued over a time interval $[t_0, t_1]$:*

$$F_m(t_0, t_1) = \left\lfloor \frac{\mathcal{A}_v(t_1) \cdot (t_1 - t_0) \cdot f_m}{\text{SECS\_PER\_YEAR} \cdot \text{MAX\_BPS}} \right\rfloor \tag{32}$$

where $t_0 = t_{\text{lastMgmtCharge}}$ is the timestamp of the last management fee processing and t_1 is the current evaluation time.

Property 5.1 (Linear Time Prorating)**.** *Management fees are proportional to the time elapsed and the assets under management. For an annualized rate of f_m bp, the fee for a period Δt seconds is:*

$$F_m = \mathcal{A} \cdot \frac{f_m}{\text{MAX\_BPS}} \cdot \frac{\Delta t}{\text{SECS\_PER\_YEAR}} \tag{33}$$

5.2 Performance Fees

Definition 5.2 (High-Water Mark)**.** *The high-water mark $\hat{\pi}_v$ records the highest net share price ever achieved by vault v. It is updated after each performance fee charge:*

$$\hat{\pi}_v \leftarrow \max\big(\hat{\pi}_v,\ \pi_v^{\text{net,post}-\text{fees}}(t)\big) \tag{34}$$

where $\pi_v^{\text{net,post}-\text{fees}}$ is computed using fee-adjusted total assets.

Definition 5.3 (Performance Fee Computation)**.** *Given performance fee rate f_p (bp), hurdle rate h (bp), and the high-water mark $\hat{\pi}_v$, the performance fee for period $[t_0, t_1]$ is computed as follows.*

Step 1: Compute reference total assets at watermark.

$$\mathcal{A}^{\text{ref}} = \left\lfloor \frac{\mathcal{S}_v \cdot \hat{\pi}_v}{10^{d_v}} \right\rfloor \tag{35}$$

Step 2: Compute post-management-fee total assets.

$$\mathcal{A}^{\text{post}} = \mathcal{A}_v - F_m \tag{36}$$

Step 3: Compute profit since watermark.

$$\Delta = \mathcal{A}^{\text{post}} - \mathcal{A}^{\text{ref}} \tag{37}$$

Step 4: If $\Delta > 0$, compute hurdle return.

$$R_h = \left\lfloor \frac{\mathcal{A}^{\text{ref}} \cdot h \cdot (t_1 - t_0^{\text{perf}})}{\text{SECS\_PER\_YEAR} \cdot \text{MAX\_BPS}} \right\rfloor \tag{38}$$

where t_0^{perf} is the timestamp of the last performance fee processing.

Step 5: If $\Delta > R_h$ (total return exceeds hurdle):

$$R_e = \Delta - R_h \quad \textit{(excess return above hurdle)} \tag{39}$$

$$F_p = \begin{cases} \left\lfloor \dfrac{R_e \cdot f_p}{\text{MAX\_BPS}} \right\rfloor & \textit{if hard hurdle} \\ \left\lfloor \dfrac{\Delta \cdot f_p}{\text{MAX\_BPS}} \right\rfloor & \textit{if soft hurdle} \end{cases} \tag{40}$$

Otherwise: $F_p = 0$ *(no performance fee if returns do not exceed hurdle or if there is a loss).*

Remark 5.1 (Hard vs. Soft Hurdle)**.**

- ***Hard hurdle****: Performance fee is charged* only on the excess return above the hurdle*. This is more favorable to users.*
- ***Soft hurdle****: Once total return exceeds the hurdle, the performance fee is charged* on the entire return Δ*. This is more favorable to the vault operator.*

5.3 Total Fees

Definition 5.4 (Total Accrued Fees)**.**

$$F_{\text{total}}(t) = F_m\big(t_{\text{lastMgmt}}, t\big) + F_p\big(t_{\text{lastPerf}}, t\big) \tag{41}$$

Invariant 5.1 (Fee Non-Negativity)**.** *Fees are always non-negative:*

$$F_m \geq 0 \quad \textit{and} \quad F_p \geq 0 \quad \textit{and} \quad F_{\text{total}} \geq 0 \tag{42}$$

Performance fees are zero when the vault is below its high-water mark or returns do not exceed the hurdle rate.

Invariant 5.2 (Fee Boundedness)**.** *Total fees never exceed gross total assets:*

$$F_{\text{total}}(t) \leq \mathcal{A}_v(t) \tag{43}$$

This ensures net assets remain non-negative.

6 Settlement Mechanism

Settlement is the process by which off-chain yield changes are reflected on-chain. It operates through a four-phase pipeline: **Proposal** → **Cooldown** → **Approval** (conditional) → **Execution**.

6.1 Proposal Phase

A relayer proposes a settlement with the observed total assets $\mathcal{A}_{\text{reported}}$ from the external strategy.

Definition 6.1 (Settlement Proposal)**.** *For a vault v with asset a and batch b:*

$$\mathcal{A}_{\text{last}} = \mathcal{V}(v, a) \tag{44}$$

$$Y = \mathcal{A}_{\text{reported}} - \mathcal{A}_{\text{last}} \tag{45}$$

$$N = \text{netted}_b \quad \textit{(from Eq. 17 or 31)} \tag{46}$$

$$\mathcal{A}_{\text{adjusted}} = \mathcal{A}_{\text{reported}} + N \tag{47}$$

where Y is the yield, N is the net deposit/withdrawal flow, and $\mathcal{A}_{\text{adjusted}}$ is the adjusted total assets accounting for batch flows.

Definition 6.2 (Yield Tolerance Check)**.** *A proposal requires guardian approval if the yield deviates beyond the configured tolerance:*

$$\text{requiresApproval} \iff |Y| > \frac{\mathcal{A}_{\text{last}} \cdot \delta_v}{\text{MAX\_BPS}} \tag{48}$$

where δ_v is the per-vault maximum allowed delta in basis points.

Definition 6.3 (Cooldown)**.** *The proposal becomes executable after:*

$$t_{\text{execute}} = t_{\text{propose}} + T_{\text{cooldown}} \tag{49}$$

where $T_{\text{cooldown}} \in [0, T^{\max}_{\text{cooldown}}]$.

6.2 Execution: kMinter Settlement

When a kMinter batch b is settled:

1. If b.requestedSharesInBatch > 0: withdraw b.requestedSharesInBatch units from the adapter and transfer to the batch receiver.
2. Update the kMinter adapter's virtual balance:

$$\mathcal{V}(\text{kMinter}, a) \leftarrow \mathcal{V}(\text{kMinter}, a) + N \tag{50}$$

3. Mark batch as settled.

Invariant 6.1 (kMinter Adapter Balance)**.** *After kMinter settlement, the adapter's virtual balance reflects the net position change:*

$$\mathcal{V}'(\text{kMinter}, a) = \mathcal{V}(\text{kMinter}, a) + \text{netted}_b \tag{51}$$

6.3 Execution: kStakingVault Settlement

When a staking vault v with batch b is settled:

1. **Yield distribution via kToken mint/burn:**

$$\mathsf{kToken}_a.\text{totalSupply} \leftarrow \begin{cases} \mathsf{kToken}_a.\text{totalSupply} + Y & \text{if } Y > 0 \text{ (profit)} \\ \mathsf{kToken}_a.\text{totalSupply} - |Y| & \text{if } Y < 0 \text{ (loss)} \end{cases} \tag{52}$$

   This maintains the invariant that kToken supply reflects true asset backing.

2. **Fee notification:** The vault is notified of charged management and performance fees via timestamp updates, triggering watermark recalculation.

3. **Vault internal settlement:** The vault snapshots state for claim calculations:

$$b.\text{totalAssets} \leftarrow \mathcal{A}_v \tag{53}$$
$$b.\text{totalNetAssets} \leftarrow \mathcal{A}_v^{\text{net}} \tag{54}$$
$$b.\text{totalSupply} \leftarrow \mathcal{S}_v \tag{55}$$

4. **Share minting for stakers:** If $b.\text{depositedInBatch} > 0$:

$$s_{\text{new}} = \text{toShares}\big(b.\text{depositedInBatch},\ \mathcal{A}_v^{\text{net}},\ \mathcal{S}_v\big) \tag{56}$$

   These s_{new} shares are minted to the vault itself, to be distributed to individual claimants proportionally.

5. **Share burning for unstakers:** If $b.\text{requestedSharesInBatch} > 0$:

$$\text{grossKTokens} = \text{toAssets}_{\text{exact}}\big(b.\text{requestedSharesInBatch},\ \mathcal{A}_v,\ \mathcal{S}_v\big) \tag{57}$$
$$\text{netKTokens} = \text{toAssets}_{\text{exact}}\big(b.\text{requestedSharesInBatch},\ \mathcal{A}_v^{\text{net}},\ \mathcal{S}_v\big) \tag{58}$$
$$\text{feeKTokens} = \text{grossKTokens} - \text{netKTokens} \tag{59}$$

   The $b.\text{requestedSharesInBatch}$ shares are burned; feeKTokens are transferred to the treasury; netKTokens are reserved for user claims.

6. **Adapter update:**

$$\mathcal{V}(v, a) \leftarrow \mathcal{A}_{\text{adjusted}} \tag{60}$$

7. **kMinter adapter adjustment** (counterpart of virtual transfer):

$$\mathcal{V}(\text{kMinter}, a) \leftarrow \mathcal{V}(\text{kMinter}, a) - N \tag{61}$$

8. **Global pending cleanup:**

$$\mathcal{G}(\text{kMinter}, a) \leftarrow \mathcal{G}(\text{kMinter}, a) - b.\text{depositedInBatch} \tag{62}$$

6.4 Claim Phase

Definition 6.4 (Stake Claim)**.** *A user who deposited x kTokens in batch b claims:*

$$s_{\text{user}} = \text{toShares}\big(x,\ b.\text{totalNetAssets},\ b.\text{totalSupply}\big) \tag{63}$$

stkTokens from the vault's pre-minted pool.

Definition 6.5 (Unstake Claim)**.** *A user who unstaked s stkTokens in batch b claims:*

$$x_{\text{user}} = \text{toAssets}\big(s,\ b.\text{totalNetAssets},\ b.\text{totalSupply}\big) \tag{64}$$

kTokens from the vault's reserved pool, reducing Ψ_v^-.

7 Virtual Balance and Effective Balance

Definition 7.1 (Virtual Balance)**.** *The virtual balance of vault v for asset a is the total assets reported by the adapter:*

$$\mathcal{V}(v, a) = \text{adapter}(v, a).\text{totalAssets}() \tag{65}$$

Definition 7.2 (Effective Virtual Balance)**.** *The effective virtual balance accounts for all pending settlement proposals:*

$$\mathcal{V}^{\text{eff}}(v, a) = \mathcal{V}(v, a) + \sum_{\substack{p \in \text{pending}(v) \\ p.\text{asset}=a}} p.\text{netted} \tag{66}$$

Invariant 7.1 (Virtual Balance Sufficiency)**.** *At all times, the effective virtual balance must cover all global pending requests:*

$$\mathcal{V}^{\text{eff}}(v, a) \geq \mathcal{G}(v, a) \geq 0 \tag{67}$$

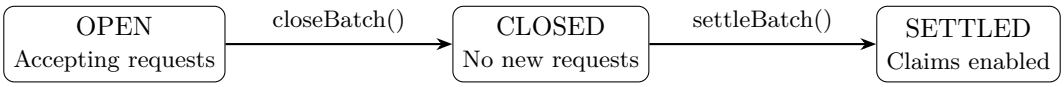
This ensures the protocol can always honor outstanding redemption and transfer obligations.

Invariant 7.2 (Non-Negative Effective Balance)**.** *The effective virtual balance must never go negative:*

$$\mathcal{V}^{\text{eff}}(v, a) \geq 0 \tag{68}$$

8 Batch Lifecycle

Each batch b progresses through a strict lifecycle:



Invariant 8.1 (Batch Monotonicity)**.** *Batch state transitions are monotonic and irreversible:*

$$\mathit{OPEN} \rightarrow \mathit{CLOSED} \rightarrow \mathit{SETTLED} \tag{69}$$

Once a batch is closed, no new requests can be added. Once settled, settlement cannot be repeated.

Invariant 8.2 (Batch Isolation)**.** *Deposits and requests within a batch are immutable after closure:*

$$b.\text{isClosed} = \mathsf{true} \implies b.\text{depositedInBatch} = \mathit{const} \tag{70}$$

$$b.\text{isClosed} = \mathsf{true} \implies b.\text{requestedSharesInBatch} = \mathit{const} \tag{71}$$

9 System Invariants

We now enumerate the critical invariants that must hold for protocol correctness.

9.1 Conservation Invariants

Invariant 9.1 (Asset Conservation)**.** *For each asset a, the total kToken supply equals the sum of all underlying assets controlled by the protocol (across adapters, batch receivers, and pending operations):*

$$\mathsf{kToken}_a.\text{totalSupply} = \sum_v \mathcal{V}(v, a) + \sum_b \text{receiver}(b).\text{balance}(a) + \mathit{pending\ adjustments} \tag{72}$$

Invariant 9.2 (Share-Asset Proportionality)**.** *For any staking vault v, the stkToken shares represent proportional claims on net assets. For any two shareholders holding s_1 and s_2 shares:*

$$\frac{\text{claim}(s_1)}{\text{claim}(s_2)} = \frac{s_1}{s_2} \tag{73}$$

9.2 Solvency Invariants

Invariant 9.3 (Vault Solvency)**.** *Each staking vault must have sufficient kTokens to cover all obligations:*

$$\mathsf{kToken}.\text{balanceOf}(v) \geq \Psi_v^+ + \Psi_v^- \tag{74}$$

Invariant 9.4 (Settlement Solvency)**.** *Before settlement execution, the adapter must hold sufficient assets to cover all batch redemptions:*

$$\text{adapter.actualBalance}(a) \geq b.\text{requestedSharesInBatch} \tag{75}$$

for kMinter batches (where requests are denominated in asset units directly).

9.3 Watermark Invariants

Invariant 9.5 (Watermark Monotonicity)**.** *The high-water mark never decreases during performance fee processing:*

$$\hat{\pi}_v(t_2) \geq \hat{\pi}_v(t_1) \quad \forall\, t_2 > t_1 \tag{76}$$

It is updated to $\max(\hat{\pi}_v, \pi_v^{\text{net,post}-\text{fees}})$ *after each performance fee charge.*

Invariant 9.6 (No Double Fee Charging)**.** *Performance fees are only charged on profit above the watermark. After charging, the watermark advances to the current share price, ensuring the same profit is never taxed twice:*

$$\hat{\pi}_v^{\text{new}} \geq \pi_v^{\text{net,post}-\text{fees}}(t_{\text{charge}}) \tag{77}$$

9.4 Timing Invariants

Invariant 9.7 (Fee Timestamp Progression)**.** *Fee charge timestamps always progress forward:*

$$t_{\text{lastMgmt}}^{\text{new}} \geq t_{\text{lastMgmt}}^{\text{old}} \tag{78}$$

$$t_{\text{lastPerf}}^{\text{new}} \geq t_{\text{lastPerf}}^{\text{old}} \tag{79}$$

$$t_{\text{lastMgmt}}^{\text{new}} \leq t_{\text{current}} \tag{80}$$

$$t_{\text{lastPerf}}^{\text{new}} \leq t_{\text{current}} \tag{81}$$

This prevents retroactive fee manipulation.

Invariant 9.8 (Settlement Cooldown)**.** *A settlement proposal cannot be executed before its cooldown period elapses:*

$$t_{\text{execute}} \geq t_{\text{propose}} + T_{\text{cooldown}} \tag{82}$$

9.5 Access Control Invariants

Invariant 9.9 (Single Pending Proposal)**.** *For staking vaults, at most one settlement proposal can be pending at any time. For the kMinter, at most one proposal per asset can be pending:*

$$v \in \mathcal{V}^{\text{stk}} \implies |\text{pending}(v)| \leq 1 \tag{83}$$

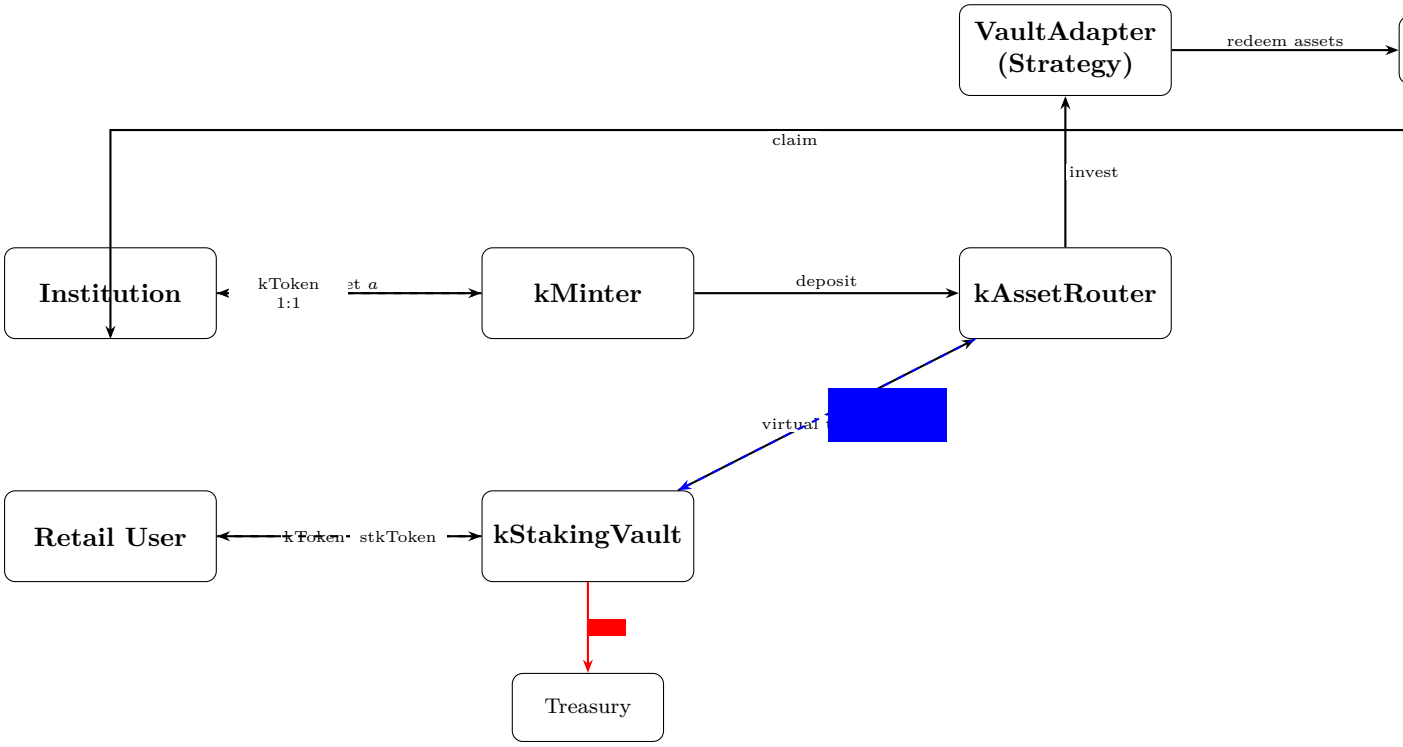
$$v = \text{kMinter} \implies |\{p \in \text{pending}(v) : p.\text{asset} = a\}| \leq 1 \quad \forall\, a \tag{84}$$

Invariant 9.10 (Proposal Idempotency)**.** *Each proposal can be executed at most once:*

$$p.\text{id} \in \text{executedSet} \implies p \text{ cannot be re-executed} \tag{85}$$

10 Money Flow Diagram

The following diagram illustrates the complete asset flow through the protocol:



Solid arrows: asset/token transfers. **Dashed arrows**: token minting or virtual operations. **Blue**: yield distribution. **Red**: fee extraction.

11 Settlement Mathematics: Worked Example

Consider a staking vault v for USDC ($d_v = 6$) with the following state:

Parameter	Value
$\mathcal{A}_v$	1,000,000 USDC
$\mathcal{S}_v$	950,000 stkUSDC
$\hat{\pi}_v$	1.05×10^6
f_m	200 bp (2% annual)
f_p	2000 bp (20%)
h	500 bp (5% annual hurdle)
Hard hurdle	Yes
Δt_m	2,592,000 s (30 days)
Δt_p	7,776,000 s (90 days)
Batch deposits	100,000 kUSDC
Batch unstake requests	50,000 stkUSDC
Reported total assets	1,050,000 USDC

11.1 Step 1: Fee Computation

Management fee:

$$F_m = \left\lfloor \frac{1{,}000{,}000 \times 2{,}592{,}000 \times 200}{31{,}556{,}952 \times 10{,}000} \right\rfloor = \lfloor 1{,}642.01 \rfloor = 1{,}642 \text{ USDC} \tag{86}$$

Performance fee:

$$\mathcal{A}^{\text{ref}} = \left\lfloor \frac{950{,}000 \times 1{,}050{,}000}{1{,}000{,}000} \right\rfloor = 997{,}500 \tag{87}$$

$$\mathcal{A}^{\text{post}} = 1{,}000{,}000 - 1{,}642 = 998{,}358 \tag{88}$$

$$\Delta = 998{,}358 - 997{,}500 = 858 \tag{89}$$

$$R_h = \left\lfloor \frac{997{,}500 \times 500 \times 7{,}776{,}000}{31{,}556{,}952 \times 10{,}000} \right\rfloor = \lfloor 12{,}294.8 \rfloor = 12{,}294 \tag{90}$$

Since $\Delta = 858 < R_h = 12{,}294$, returns do not exceed the hurdle rate:

$$F_p = 0 \tag{91}$$

Total fees: $F_{\text{total}} = 1{,}642$ USDC.

11.2 Step 2: Netting

$$\text{requestedAssets} = \left\lfloor \frac{50{,}000 \times 1{,}050{,}000}{950{,}000} \right\rfloor = 55{,}263 \tag{92}$$

$$N = 100{,}000 - 55{,}263 = 44{,}737 \quad \text{(net inflow)} \tag{93}$$

11.3 Step 3: Yield and Adjusted Assets

$$Y = 1{,}050{,}000 - \mathcal{V}(v, \text{USDC}) \tag{94}$$

$$\mathcal{A}_{\text{adjusted}} = 1{,}050{,}000 + 44{,}737 = 1{,}094{,}737 \tag{95}$$

11.4 Step 4: Settlement Execution

The vault mints Y kTokens (if $Y > 0$), snapshots state, mints new stkTokens for stakers at the settlement-time share price, burns stkTokens for unstakers, and transfers fee kTokens to the treasury.

12 Security Properties

Property 12.1 (MEV Resistance via Batching)**.** *By processing all requests in a batch at the same settlement-time share price, the protocol eliminates front-running and sandwich attacks. All stakers in a batch receive the same conversion rate.*

Property 12.2 (Guardian Veto Power)**.** *Any settlement proposal can be cancelled by a guardian or emergency admin during the cooldown period. High-yield proposals ($|Y| > \delta_v \cdot \mathcal{A}_{\text{last}}/\text{MAX\_BPS}$) additionally require explicit guardian approval.*

Property 12.3 (Adapter Caching)**.** *The adapter address is cached at proposal creation time, preventing registry modifications from breaking pending settlements. This ensures settlement determinism.*

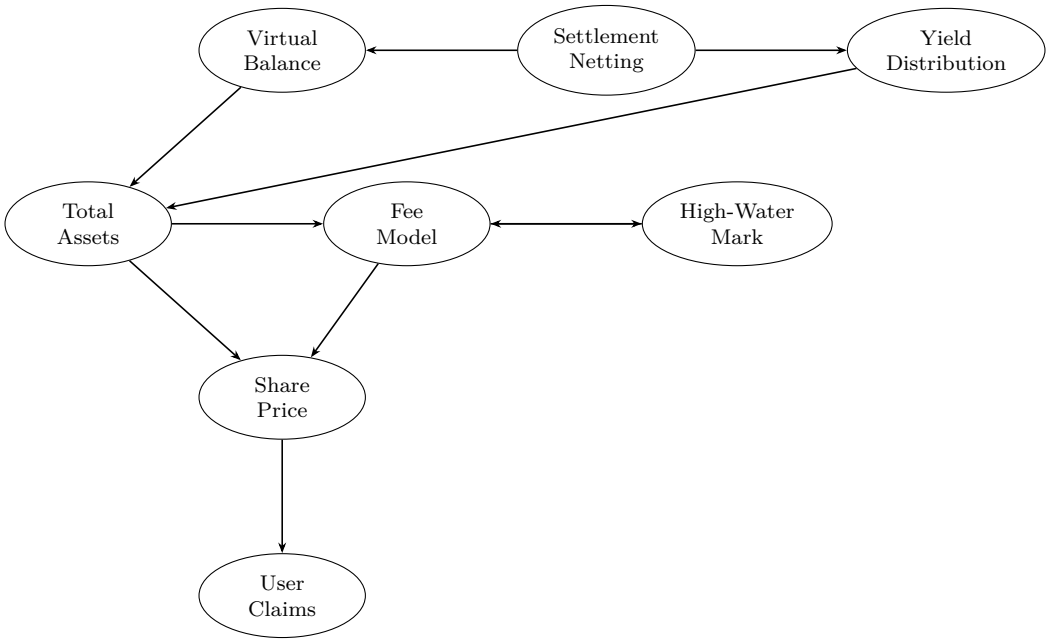
Property 12.4 (Commutative Adapter Updates)**.** *For the kMinter adapter, virtual balance updates use delta operations (+N) rather than absolute sets, making concurrent kMinter and kStakingVault settlements commutative (order-independent).*

Property 12.5 (Cross-Batch Over-Request Prevention)**.** *The $\mathcal{G}$ tracking spans all open batches, preventing users from requesting more assets than the protocol can fulfill even across multiple concurrent batches.*

13 Interconnection Summary

The mathematical components of KAM are tightly coupled:

1. **Share pricing** (Section 4.2) depends on **total assets** (Eq. 18), which depends on **pending operations** (Ψ^+, Ψ^-).
2. **Total net assets** (Eq. 19) depends on **fee computation** (Section 5), which in turn depends on total assets and the high-water mark.
3. **Settlement netting** (Eqs. 17, 31) drives **virtual balance updates** (Section 7), which constrain future **request acceptance** via Invariant 7.1.
4. **Yield distribution** (Eq. 52) adjusts the **kToken supply**, which feeds back into **total assets** for staking vaults (Eq. 18).
5. **Fee extraction** at settlement time creates a spread between gross and net share prices, directly affecting **claim amounts** (Eqs. 63, 64) and the **watermark** (Definition 5.2).
6. The **watermark** determines whether **performance fees** apply (Definition 5.3), creating a feedback loop with net assets.
7. **Global pending requests** ($\mathcal{G}$) link kMinter redemptions and staking vault transfers to **effective virtual balance** (Definition 7.2), ensuring cross-vault solvency.



14 Conclusion

The KAM protocol's mathematical foundation rests on several interlocking systems:

- **1:1 institutional minting** with batch-processed redemption ensures capital efficiency while maintaining the backing guarantee.
- **Share-based staking** with ERC-4626 virtual offsets provides inflation-attack-resistant retail yield access.
- **Two-component fee model** (management + performance with hurdle rates and high-water marks) aligns incentives between vault operators and users.
- **Timelocked settlement proposals** with guardian oversight provide security against erroneous or malicious yield reports.
- **Virtual balance accounting** with effective balance computation prevents over-commitment across concurrent batches and vaults.

The 17 invariants enumerated in this document form the correctness specification of the protocol. Any violation of these invariants would indicate a protocol-level bug or potential exploit vector. Formal verification efforts should target these invariants as their primary proof obligations.